

Concurrent Task Dispatcher: Design Report

CS3334 Systems Programming — Final Project — Lewis (LuisJC)

Overview

This project is a multithreaded task dispatcher in Rust. The simulation generates 1000 tasks that arrive every 20ms and runs them across 8 worker threads under a 100% CPU usage cap. Tasks are split 70/30 between IO (10% load, 200ms each) and CPU (35% load, 200ms each). I compare two scheduling policies. Experiment A is a basic FIFO that takes the front of the queue when it fits the cap. Experiment B prefers CPU tasks and falls back to the head IO task when no CPU task fits. ExpB cuts total runtime by about 7%.

Architecture

There are three thread roles. The sender thread generates the deterministic task stream (seed 42) and pushes tasks into a shared queue at fixed wall-clock arrival times. I use absolute timing ($\text{target} = \text{start} + i * 20\text{ms}$) instead of relative sleeps so that lock contention does not push later arrivals out of schedule. After the last task, the sender pushes one None sentinel per worker so the worker pool can drain cleanly.

The worker pool is 8 OS threads. Each worker locks the queue and the load counter, decides whether a task can be admitted, updates the load if yes, releases both locks, sleeps for the task's duration, then re-locks the load counter to subtract the task's cost. Workers exit when they pull a None from the queue. They never hold any lock during the 200ms simulated work, which keeps the dispatch path short and lets other workers run in parallel.

The monitor thread wakes every 10ms, reads the load counter and the active-worker count, and appends a snapshot to a shared vector. After all workers finish, the main thread averages the snapshots and prints the metrics.

Synchronization

Three pieces of shared state are protected by three different primitives. The task queue is an `Arc<Mutex<VecDeque<Option<Task>>>>`. The Option lets the sender push None as a shutdown signal without a separate channel. The CPU load counter is an `Arc<Mutex<f64>>`. The mutex must be held across the read, the cap check, and the increment so two workers cannot both decide there is room when only one slot fits. The active-worker count is an `Arc<AtomicUsize>` because it is updated on the worker hot path and read by the monitor with no other invariants attached.

Workers always acquire the queue lock and the load lock in the same scope at the dispatch decision and release them together before the simulated work. Using a fixed acquisition order eliminates the possibility of a deadlock cycle and makes the dispatch code small enough to reason about by inspection.

Scheduling Policies

ExpA — basic FIFO. A worker looks at the front of the queue and checks $\text{current_load} + \text{task.cpu_cost} \leq 100$. If it fits, dispatch it. If not, sleep 1ms and retry. When the front is a CPU task and the load is already high, every worker blocks on that head until enough load drains, even when smaller IO tasks behind it would have fit.

ExpB — CPU-preferring. A worker first scans the queue for the first CPU task that fits. If found, dispatch that task. If no CPU task fits (none in queue, or load too high), fall back to the head and dispatch the IO task there if it fits. The idea is that a CPU task at 35% load each represents 3.5x the work of a 10% IO task, so prioritizing CPU tasks means each utilization slot does more work and the heavier portion of the workload drains earlier. The trade-off is that IO tasks wait longer because they get skipped over whenever a fittable CPU task exists.

Metrics

Each binary records and prints these metrics at the end of the run. Makespan is the wall-clock time from start to last completion. Avg CPU consumption is the time-weighted average of the load counter sampled every 10ms. Avg active workers is the same for the worker counter. Wait time is $\text{start_time} - \text{end_time}$.

arrival_time per task, with separate averages by task kind to show how each policy treats CPU and IO differently. Max wait time is the worst case across the 1000 tasks.

Experiments

Both runs use the same workload. Only the dispatch decision changes between them.

Metric	ExpA (FIFO)	ExpB (CPU-preferring)	Change
Makespan	39079 ms	36343 ms	-2736 ms (-7.0%)
Avg CPU	89.48 %	96.23 %	+6.75 pp
Avg active workers	5.12	5.51	+0.39
Avg wait (all)	9205.98 ms	9461.40 ms	+255 ms
Avg wait (CPU)	9528.33 ms	4725.83 ms	-4802 ms (-50.4%)
Avg wait (IO)	9068.49 ms	11481.28 ms	+2413 ms (+26.6%)
Max wait	18915 ms	17352 ms	-1563 ms

ExpB beats ExpA by 2.7 seconds on makespan. The mechanism is visible in the other numbers. Avg CPU went from 89% to 96%, meaning the cap is being kept fuller. Avg active workers went from 5.12 to 5.51, meaning workers idle less. CPU tasks wait half as long because workers actively pull them out of the queue. IO tasks wait 26% longer because they get skipped over when a CPU task fits. Max wait still drops, so the policy isn't producing a long tail in exchange for a better average. Total CPU-task load ($300 \times 35 = 10500$ load-units) outweighs total IO load ($700 \times 10 = 7000$), so draining CPU faster shortens the post-arrival cleanup phase, which is where ExpA spends most of its time.

Lessons Learned

The biggest lesson was that architecture changes did not move the metric. Before landing on the working ExpB, I tried partitioned worker pools (2 dedicated CPU workers and 6 IO workers), a hybrid pool with a CPU-preferring "floater" worker, scan-and-fit policies that took the first task that fits regardless of type, single-mutex dual-queue structures, and an mpvc channel rewrite. None of them beat the FIFO baseline. They all ran around 41-42 seconds. The version that finally won was a two-line policy change on the same single-queue structure as ExpA — just "prefer CPU when one fits, else take the head IO." The architecture wasn't the problem. The dispatch policy was.

A bug worth mentioning: an earlier version checked queue length inside the worker's idle loop, separately from the dispatch path, to decide when to exit. Under load this raced with another worker mid-pop and a worker would sometimes exit while the queue was nonempty. The fix was to drop the length-based termination and use the None sentinel pattern instead. The sender pushes one None per worker after the last real task. Each worker exits when it pulls a None. Since None goes through the same queue mutex as real tasks, there's no race.